

LAB 1: Implementing a Class

DOCUMENTATION

The goal of this lab is to implement two classes with their attributes and methods and a test to make sure that each class and all their methods work how they are supposed to and the program does not give any errors.

We had to define two classes:

- **Vector:** represents a mathematical vector of doubles and can do basic math
 - **Attributes:** double[] elems - *encapsulated*
 - **Constructors:**
 - Vector(double[] e) - *from existing array*
 - Vector(int dim, double val) - *dimension of the array, filled with value*
 - **Getters:**
 - int getDim() - *gets the dimension of the vector.*
 - double[] getElems() - *we also created a getter for the elements array to be able to access them.*
 - **Methods (operations):**
 - Vector add(Vector v) - *adds two vectors*
 - Vector subtract(Vector v) - *subtracts two vectors*
 - Vector multiply(Vector v) - *element-wise of two vectors*
 - Vector divide(Vector v) - *element-wise of two vectors*
 - Vector multiply(Vector scalar) - *multiplication of a vector by a scalar*
 - Vector divide(double scalar) - *division of a vector by a scalar*
 - Vector sqrt() - *compute the square root of every element of a vector*
 - double dotProduct(Vector v) - *compute the dot product of two vectors*
 - double norm() - *compute the norm of a vector*
 - String toString() - *convert a vector to string for easy printing*
- **Record:** pairs an input vector with a scalar output.
 - **Attributes:**
 - Vector input
 - double output
 - **Constructor:**
 - Record(Vector i, double o)
 - **Getters:**
 - getInput() - *gets the vector input*
 - getOutput() - *gets the double output*
 - **Method:**
 - toString() → prints " [..vector..] -> output " - *convert record to string for easy printing.*

- **TestRecord:** test program (main method) to make sure both classes are programmed correctly with no errors, since a class definition does nothing on its own.
 - Instantiates several Vector / Record objects
 - Makes sure all Vector methods and Record getters/printing work
 - Counts and prints any failures or errors.
 - How we actually did it was constructing two vectors: $v1 = [1,2,3]$, $v2 = [1,1,1]$.
 - And verifying:
 - $\text{getDim}() \Rightarrow 3$.
 - $\text{add} \Rightarrow [2,3,4]$; $\text{subtract} \Rightarrow [0,1,2]$.
 - $\text{multiply}(\text{vector}) \Rightarrow [1,2,3]$; $\text{divide}(\text{vector}) \Rightarrow [1,2,3]$.
 - $\text{multiply}(2) \Rightarrow [2,4,6]$; $\text{divide}(2) \Rightarrow [0.5,1,1.5]$.
 - $\text{sqrt}([1,2,3]) \approx [1.0, 1.4142, 1.7321]$ (test with a tolerance).
 - $\text{dotProduct}(v1,v2) \Rightarrow 6$; $\text{norm}(v1) \Rightarrow \text{sqrt}(14)$.
 - $\text{Record}([1,2,3], 1.0)$ prints as "[1.0, 2.0, 3.0] -> 1.0"; getters return the expected values.
 - And then constructing vector instances: $r1 = \text{Record}(v1, 1)$, $r2 = \text{Record}(\text{new Vector}(\text{new double[]} \{5, 10, 15\}), 2.5)$
 - And verifying:
 - $r1.\text{getInput}().\text{equals}(v1)$
 - $r1.\text{getOutput}() == 1.0 \ \&\& \ r2.\text{getOutput}() == 2.5$
 - Finally the total number of errors is printed to the screen, ideally no errors are found.

To program or implement these classes we discussed a couple alternatives for our lab 1 code or solution. For the Vector class, we thought about changing the original vector versus returning a new vector for each operation. We chose that for every operation, the method would build a new Vector and leave the original untouched. This helps avoid rising errors and maintain invariants, and it pairs with our test checks: we throw `IllegalArgumentException` when dimensions don't match and when dividing by zero.

We also kept representation details encapsulated: the elements are stored in a private `double[]`, `getElems()` returns a clone rather than the internal array, and `toString()` provides a readable [a, b, c] view.

To pair inputs with results we used a small Record value object that aggregates a Vector input and a double output with private fields and simple getters, keeping data storage separate from computation.

For testing, we used individual method by method checks printing "works or does not work" per check so it's obvious which specific method failed.

This design reflects some Object-Oriented Programming concepts such as: abstraction (hiding the array), encapsulation (private state, cloned accessors), single responsibility (math in Vector, data pairing in Record), and an immutability-leaning style that simplifies reasoning and testing.

Emma Leroux (u251606)
Laura Murillo (u252070)

All in all, the solution we chose worked pretty well. The TestRecord runs showed that all the Vector math methods produced the expected results with 0 errors. Also, the Record getters returned exactly what was passed in and invalid uses (mismatched dimensions or division by zero) gave clear `IllegalArgumentException`s. The main doubts we had were on how to program in java since we've never used the language so we are still learning the language syntax and also getting familiar with object oriented programming.

Moreover, we had issues with running the .java files because the project's paths are rooted at the repository level, not the Lab1/ folder. Meaning the files had to be executed from the repository root. When we tried to run inside Lab1/, the JVM couldn't find classes on the classpath (e.g., `NoClassDefFoundError` / "class not found"), since the relative package paths no longer matched the directory structure. Once we launched from the main repository everything ran as expected. These were the main issues we encountered when completing this Lab 1.